Kotlin Programming

Industry Standard Programming Language



Prof. Muhammad **Rafiq**

Department of Game and Mobile Engineering

Keimyung University, Republic of Korea

Tel: +82 (53) 580 5926

Email: rafiq@kmu.ac.kr

Android App Development

UI/UX, Figma design, Android Studio, App development

Setting Up and Creating Your First "Hello, World!" App

- **Step 1: Install Android Studio**
- Android Studio is the official integrated development environment (IDE) for Android development.

- Download Android Studio:
  - Go to Android Studio's official website.
  - Click Download Android Studio and accept the terms and conditions.
  - Download the appropriate version for your OS (Windows, Mac, or Linux).

- Install Android Studio:
  - Open the downloaded file and follow the instructions to install it.
  - For Windows and macOS, you can stick with the default installation options.
  - Once installed, open Android Studio.

- **Step 2: Set Up Android Studio for Your First Project**
- **Initial Setup**:
  - When you open Android Studio for the first time, it will ask you to download SDK components. This can take some time, so be patient.
  - After the initial setup, you'll be brought to the "Welcome to Android Studio" screen.
- **Create a New Project**:
  - Click **New Project** on the welcome screen.
  - Select the **Empty View Activity** template (an activity is just one screen of the app) and click **Next**.

- Configure Your Project:
  - **Name**: Enter a name for your app, like “HelloWorld”.
  - **Package name**: Android Studio will suggest a package name, like com.example.helloworld. You can keep this or customize it.
  - **Save location**: Choose where you want to save the project on your computer.
  - **Language**: Select Kotlin. (Kotlin is the recommended language for modern Android development).
  - **Minimum API level**: Select the default option or API 21 (Lollipop) to support a wide range of devices.
  - Click Finish to create the project.

Familiarize Yourself with Android Studio

- Android Studio has several important areas:
- **Project Panel**: Shows your project's files and folders.
- **Code Editor**: The central area where you write your code.
- **Logcat**: Displays logs from your app while it runs, useful for debugging.

Create Your First "Hello, World!" App

- When you created the project, Android Studio automatically generated some code for a simple app. Now let's customize it.
- Open MainActivity.kt:
  - In the Project panel on the left, go to app > java > com.example.helloworld > MainActivity.
  - This file contains the code for the main screen of your app.
- Open activity_main.xml:
  - In the Project panel, go to app > res > layout > activity_main.xml.
  - This file defines the layout of your app's main screen.
- By default, it should contain a TextView with the text "Hello World!" This is your app's main display element.

Modify the “Hello, World!” Text

- Update the Text in activity_main.xml:In activity_main.xml, you’ll see the following code (or something similar):

```
<TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Hello World!"
    android:layout_centerInParent="true"/>
```

- Change android:text="Hello World!" to android:text="Hello, Android World!“
- Save the file. You’ve just customized the text that will display on the screen.

Step 6: Run Your App

- Now it’s time to see your app in action!
- **Set Up an Emulator**:
  - In the toolbar, click on the **Device Manager** icon.
  - Click **Create Device**, select a device type (e.g., Pixel 3), and then click **Next**.
  - Choose a system image (Android version) and click **Download** if needed. Then, click **Next** and **Finish**.
  - Click **Run** or **Play** to launch the emulator.

- **Run the App on the Emulator**:
  - Click on the **Run** button (a green triangle) at the top of Android Studio.
  - Select the emulator you just created.
  - After a few moments, the emulator will display your app with the text "Hello, Android World!"

Add New Empty View Activity

- Add Image
- Choose a resource
- Set constraints
- Add Text
- Change Text Value
- Change other properties
- Set Constraints

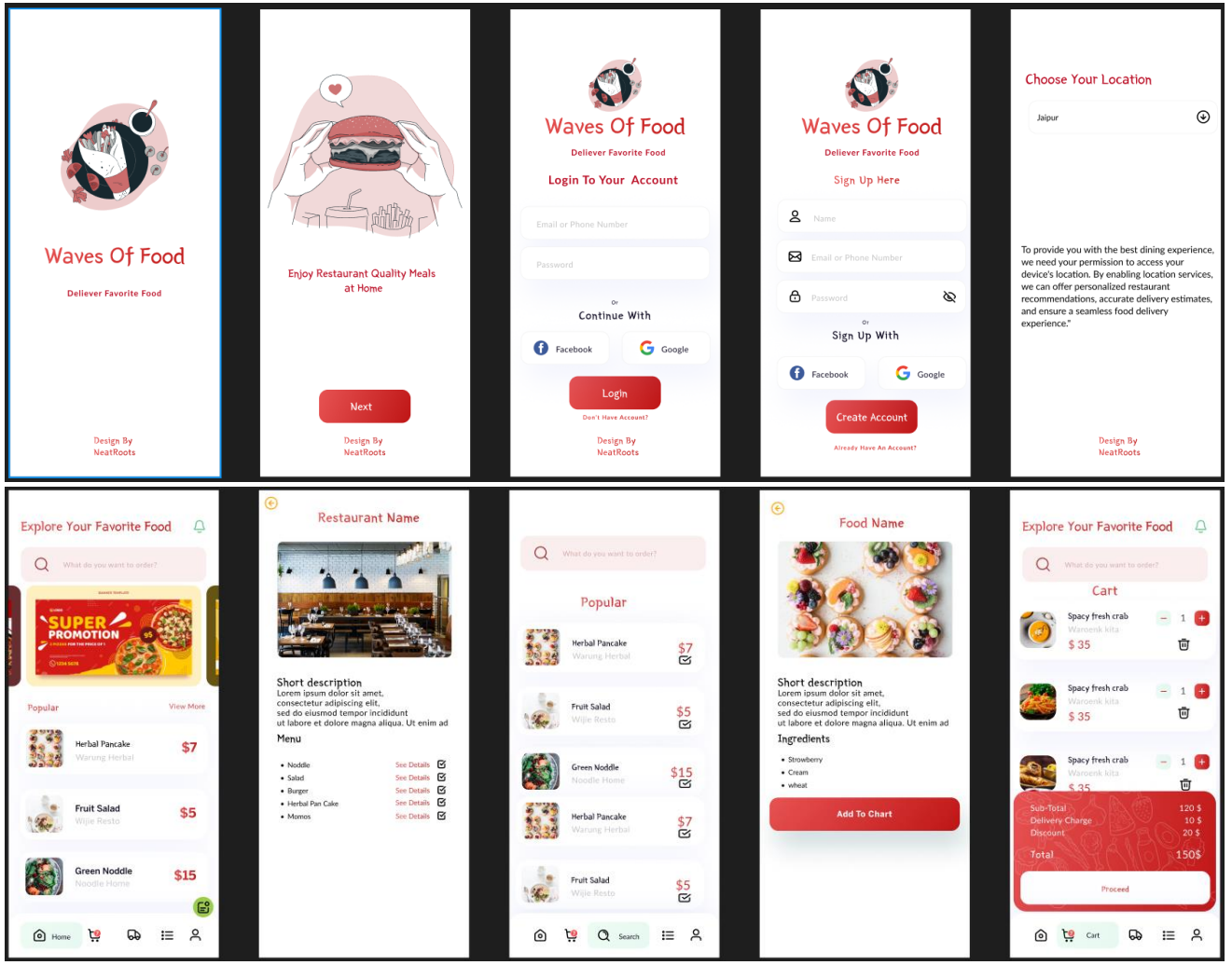
Open Figma and explore Activities

Open **Figma** Design

https://www.figma.com/design/0kKnAC2haas7qlPiqnqM1o/Green-Food-Order-App?node-id=0-1&t=G68Txl4LNSz9mXty-1

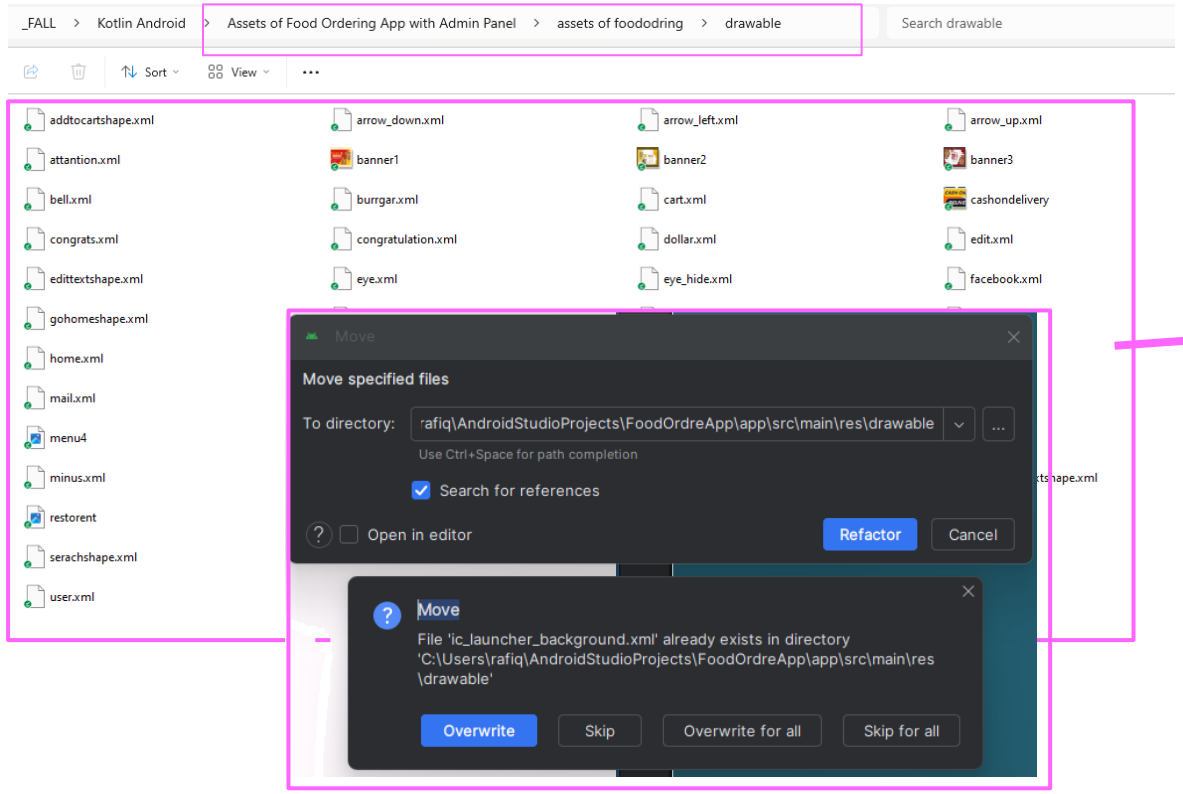
Download **drawable** Assets

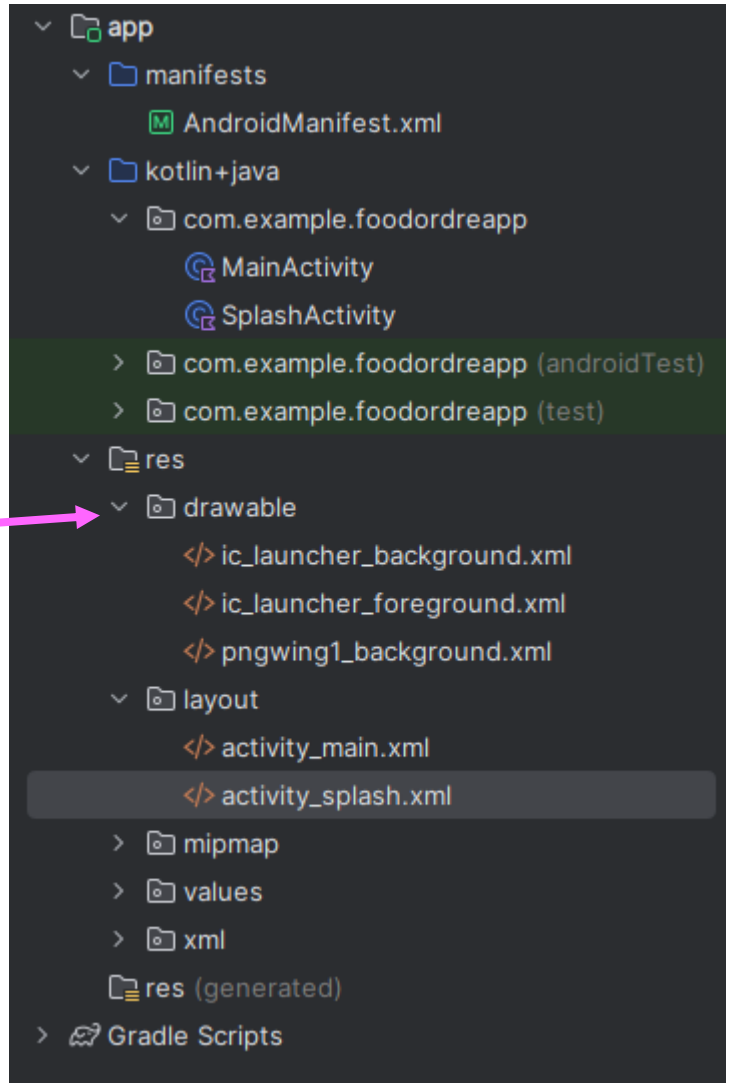
https://drive.google.com/file/d/11uIby3WWepziD447XIjsKdon5gK6wVXU/view?usp=sharing



Import Assets into project

Select All and drag drop into **drawable** in project





Confirm Refactor and Overwrite / Overwrite All

Now add custom values need in the imported assets

AddToCartShape.xml

Need 2 custom color values @color/startColor, and endColor

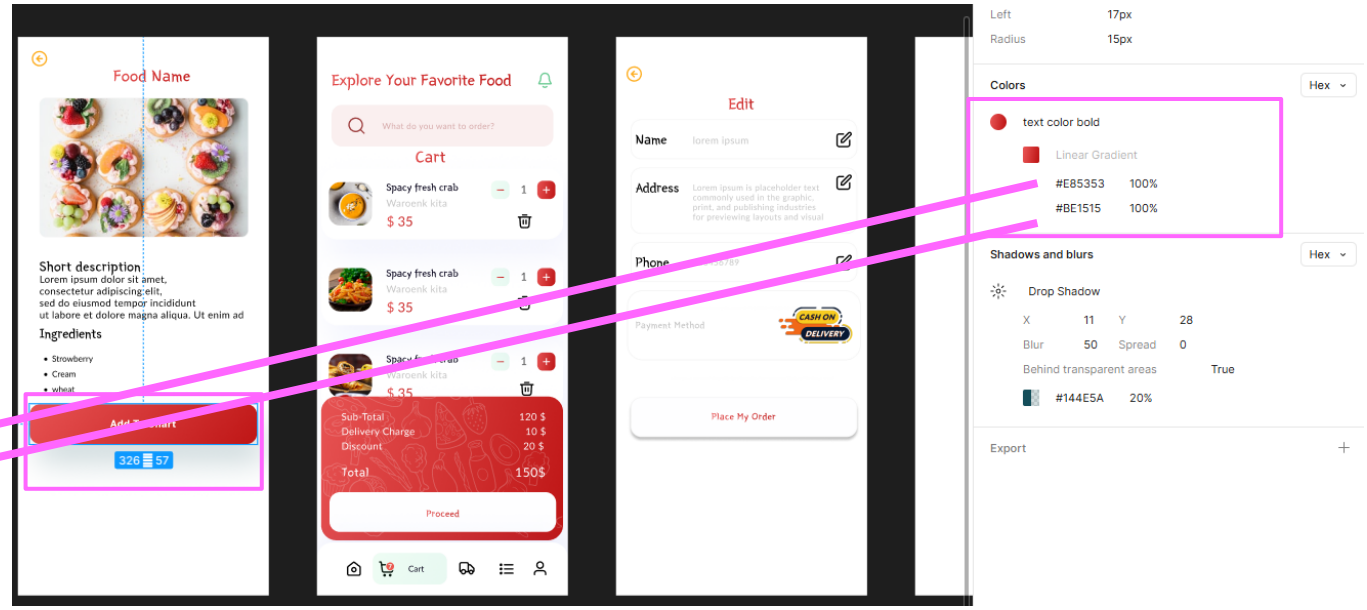
```
<?xml version="1.0" encoding="utf-8"?>
<shape xmlns:android="http://schemas.android.com/apk/res/android">
<corners android:radius="5dp"/>
    <gradient android:startColor="@color/startColor"
        android:endColor="@color/endColor" android:angle="-45"/>
</shape>
```

Open file **res/values/colors.xml** and define two colors here.

```
<resources>
    <color name="black">#FF000000<#E85353/color>
    <color name="white">#FFFFFFFF</color>
</resources>
```

Check values in figma, click button two times and read the values in right side properties

```
1  <?xml version="1.0" encoding="utf-8"?>
2  <resources>
3      <color name="black">#FF000000</color>
4      <color name="white">#FFFFFFFF</color>
5      <color name="startColor">#E85353</color>
6      <color name="endColor">#BE1515</color>
7  </resources>
```



Now add custom values need in the imported assets

arrow_left.xml

custom color @color/iconColor and textColor

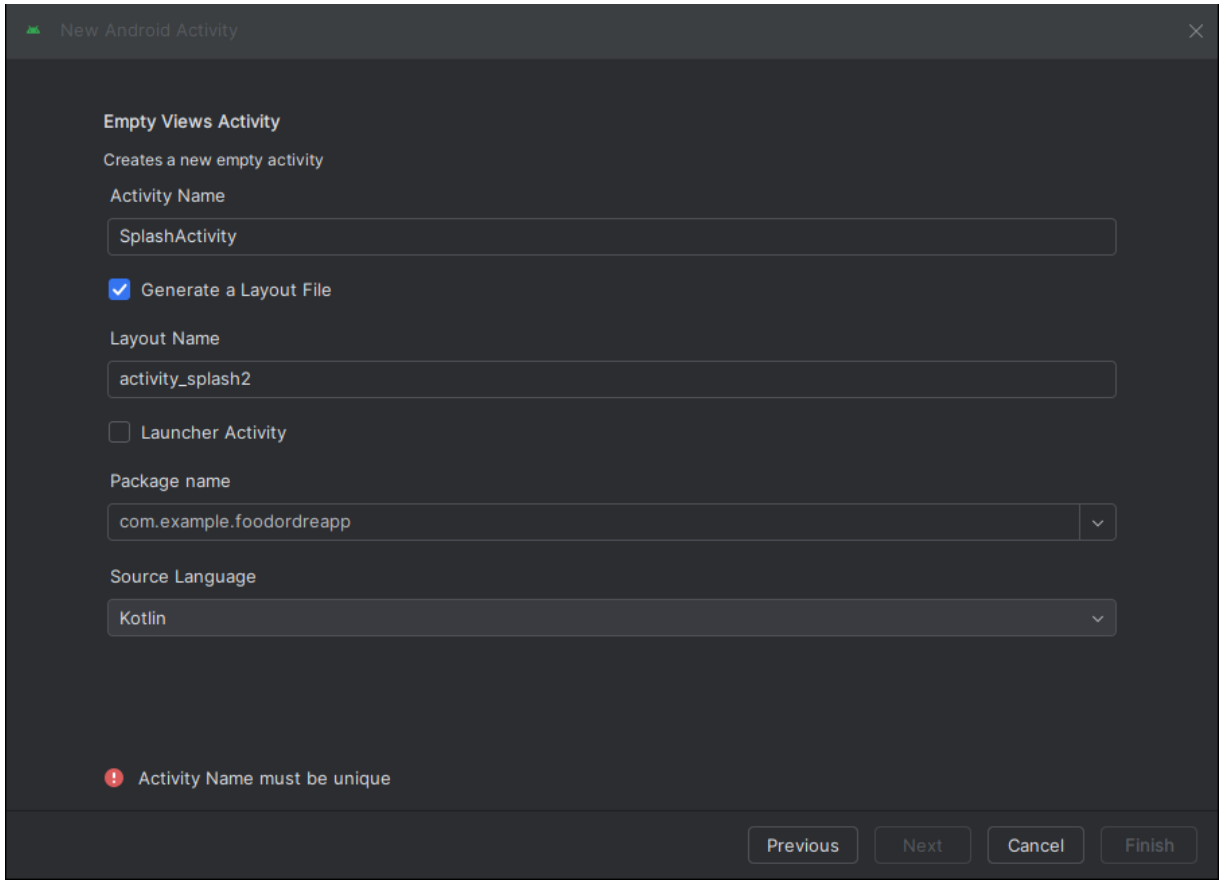


Open file **res/values/colors.xml** and define two colors here.

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <color name="black">#FF000000</color>
    <color name="white">#FFFFFFFF</color>
    <color name="startColor">#E85353</color>
    <color name="endColor">#BE1515</color>
    <color name="iconColor">#FEAD1D</color>
    <color name="textColor">#333333</color>
</resources>
```

Create Splash Activity

- Right click on the res=> layout folder
- Add Empty View Activity
- Activity Name: SplashActivity
- And Create



Make Splash open first

- Change app/manifests/AndroidManifest.xml

```
<activity
    android:name=".SplashActivity"
    android:exported="false" />
<activity
    android:name=".MainActivity"
    android:exported="true">
    <intent-filter>
        <action android:name="android.intent.action.MAIN" />

        <category android:name="android.intent.category.LAUNCHER" />
    </intent-filter>
</activity>
```



```
<activity
    android:name=".SplashActivity"
    android:exported="true">
    <intent-filter>
        <action android:name="android.intent.action.MAIN" />

        <category android:name="android.intent.category.LAUNCHER" />
    </intent-filter>
</activity>
<activity android:name=".MainActivity">
```

Run the application

- You will see splash first.
- Now add close the splash in 3 seconds and go to main activity

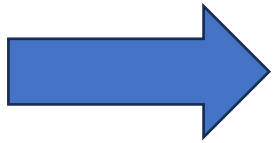
```
class SplashActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContentView(R.layout.activity_splash)
        ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main)) { v, insets ->
            val systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars())
            v.setPadding(systemBars.left, systemBars.top, systemBars.right, systemBars.bottom)
            insets
        }

        // Delay for 3 seconds, then open MainActivity
        Handler(Looper.getMainLooper()).postDelayed({
            val intent = Intent(this, MainActivity::class.java)
            startActivity(intent)
            finish() // Closes SplashActivity
        }, 3000) // 3000 milliseconds = 3 seconds

    }
}
```

Design the Splash and main activities







Open loginActivity by clicking next button

```
<!-- activity_main.xml -->
<LinearLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:gravity="center">

    <Button
        android:id="@+id/nextButton"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Next" />
</LinearLayout>
```

```
// MainActivity.kt
import android.content.Intent
import android.os.Bundle
import android.widget.Button
import androidx.appcompat.app.AppCompatActivity

class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        // Get the button and set a click listener
        val nextButton = findViewById<Button>(R.id.nextButton)
        nextButton.setOnClickListener {
            // Start LoginActivity
            val intent = Intent(this, LoginActivity::class.java)
            startActivity(intent)
        }
    }
}
```

Using binding method.

- 1. Enable View BindingIn your build.gradle file (usually under app/build.gradle), enable view binding:

```
android { ... viewBinding { enabled = true } }
```

- 2. Modify the Layout Files to Include Binding
  - Ensure you have the layout files for MainActivity and LoginActivity.activity_main.xml (with a Next button)

```
<!-- activity_main.xml -->
<LinearLayout
xmlns:android="http://schemas.android.com/apk/r
es/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:gravity="center">

    <Button
        android:id="@+id/nextButton"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Next" />
</LinearLayout>
```

```
// MainActivity.kt
import android.content.Intent
import android.os.Bundle
import androidx.appcompat.app.AppCompatActivity
import com.example.yourappname.databinding.ActivityMainBinding // Update with your package name

class MainActivity : AppCompatActivity() {

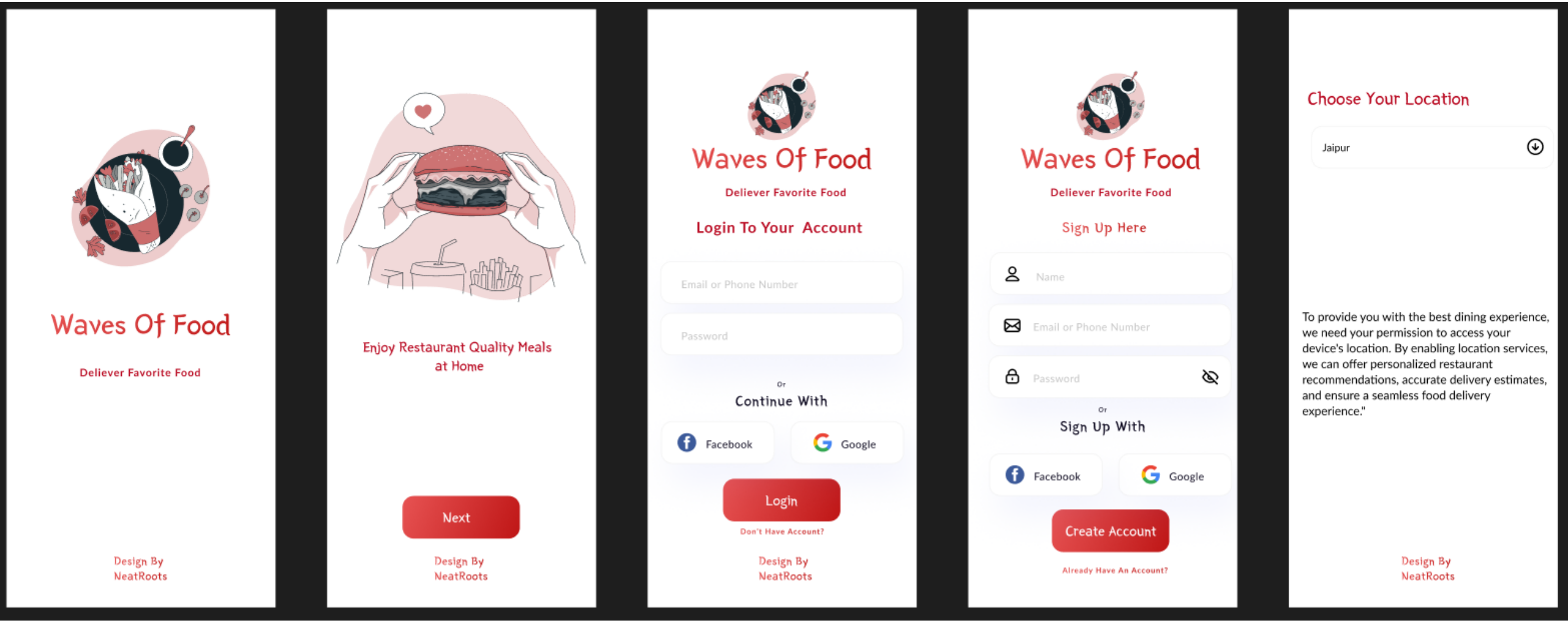
    // Declare the binding variable
    private lateinit var binding: ActivityMainBinding

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)

        // Inflate the layout using view binding
        binding = ActivityMainBinding.inflate(layoutInflater)
        setContentView(binding.root)

        // Set up the button click listener using binding
        binding.nextButton.setOnClickListener {
            // Start LoginActivity
            val intent = Intent(this, LoginActivity::class.java)
            startActivity(intent)
        }
    }
}
```

Complete the following design layouts



https://www.figma.com/design/0kKnAC2haas7qlPiqnqM1o/Green-Food-Order-App?node-id=0-1&t=G68Txl4LNSz9mXty-1

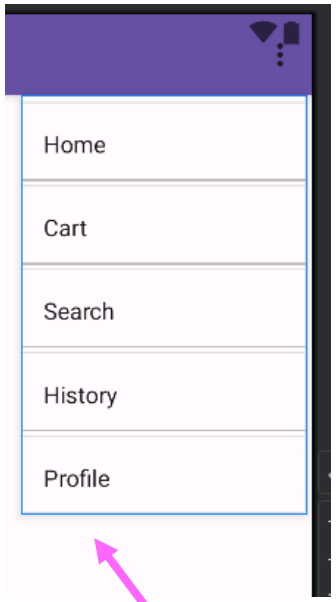
Navigation Menu

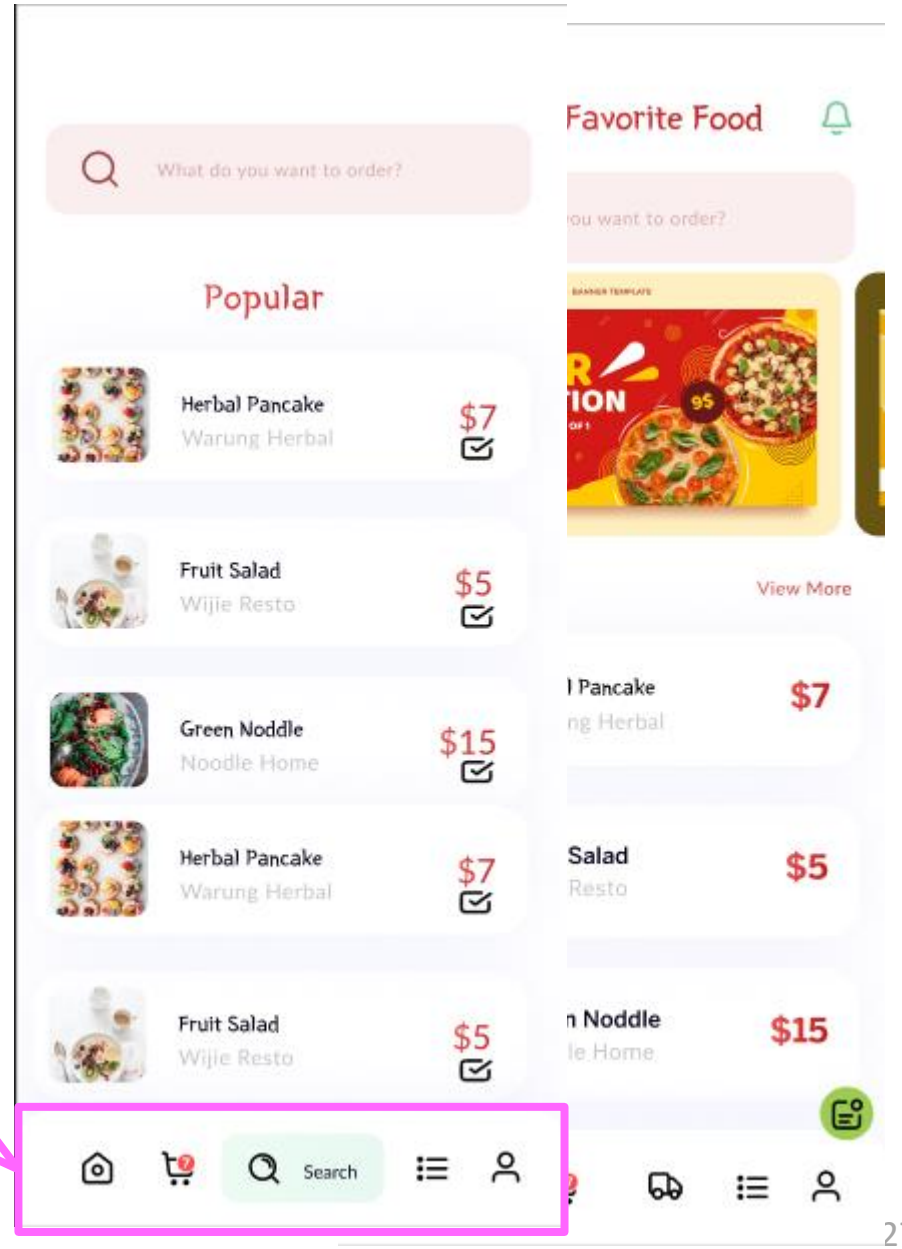
- res : Right click on res folder
  - New-> Android Resource directory
  - Resource type : menu
  - Name: menu
  - OK
- Res/menu: right click
  - New -> Menu Resource File
  - File name : menu
  - OK

menu.xml : add menu items

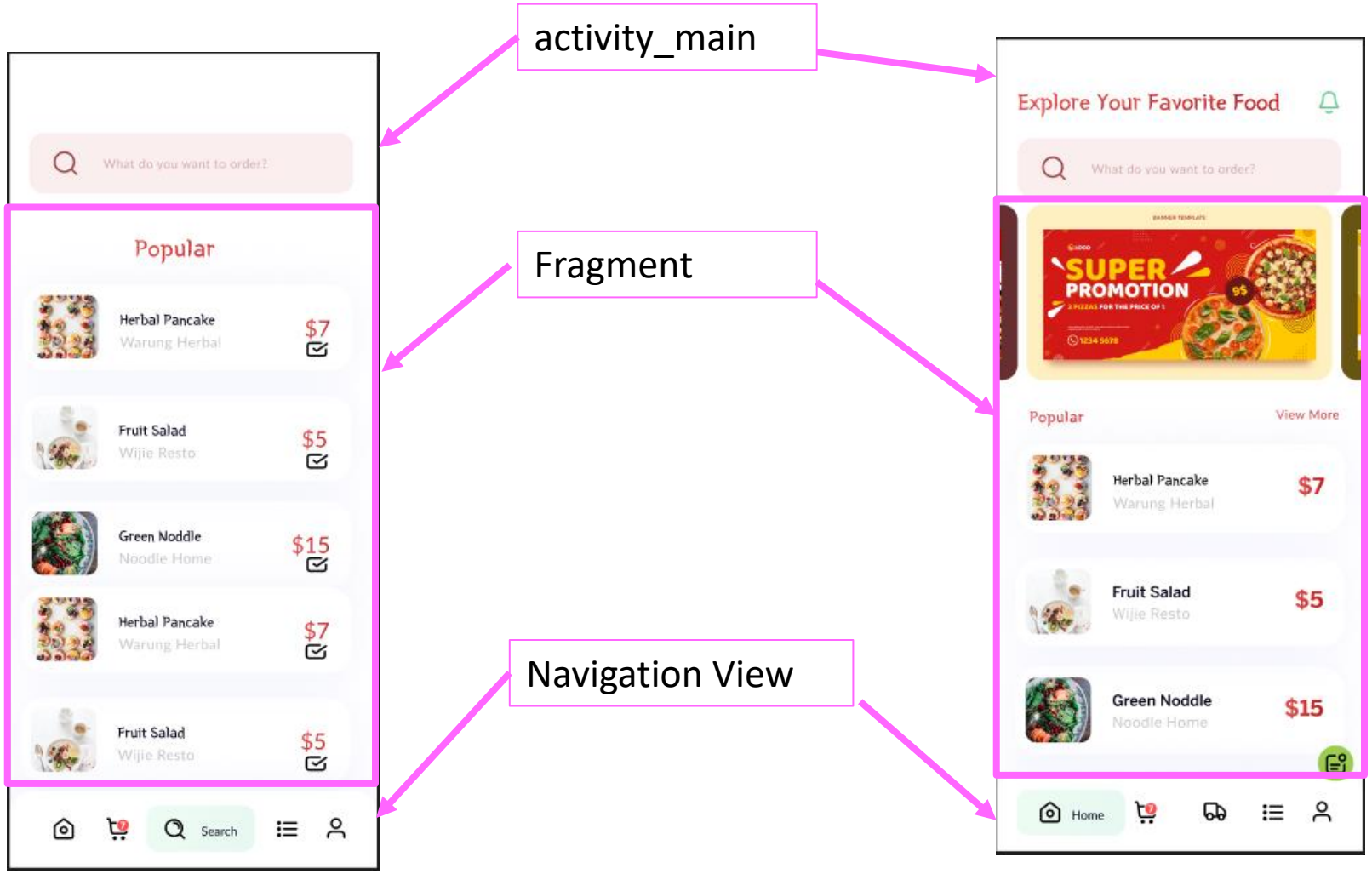
```
<?xml version="1.0" encoding="utf-8"?>
<menu xmlns:android="http://schemas.android.com/apk/res/android">

    <item android:title="Home" android:icon="@drawable/home"  />
    <item android:title="Cart"  android:icon="@drawable/cart"   />
    <item android:title="Search" android:icon="@drawable/search"  />
    <item android:title="History" android:icon="@drawable/history" />
    <item android:title="Profile" android:icon="@drawable/user"  />
</menu>
```





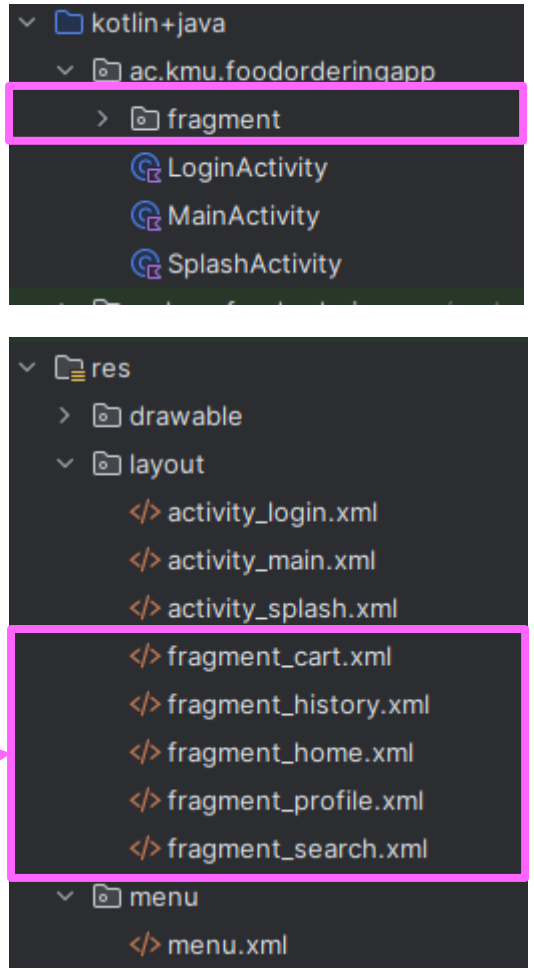
Fragment


activity_main
Fragment
Navigation View
What do you want to order?
Popular
Herbal Pancake
Warung Herbal
$7
Fruit Salad
Wijie Resto
$5
Green Noddle
Noodle Home
$15
Herbal Pancake
Warung Herbal
$7
Fruit Salad
Wijie Resto
$5
Search
Explore Your Favorite Food
What do you want to order?
SUPER
PROMOTION
Popular
View More
Herbal Pancake
Warung Herbal
$7
Fruit Salad
Wijie Resto
$5
Green Noddle
Noodle Home
$15
Home

Create Fragments

- Create Package **fragment** in code folder
  - fragment:
  - right click ->new ->Fragment-> fragment (blank)
  - Create 5 fragments
    - HomeFragment
    - CartFragment
    - SearchFragment
    - HistoryFragment
    - ProfileFragment

  - Same res/layout/fragment_***.xml are also created automatically.



menu.xml (updated)

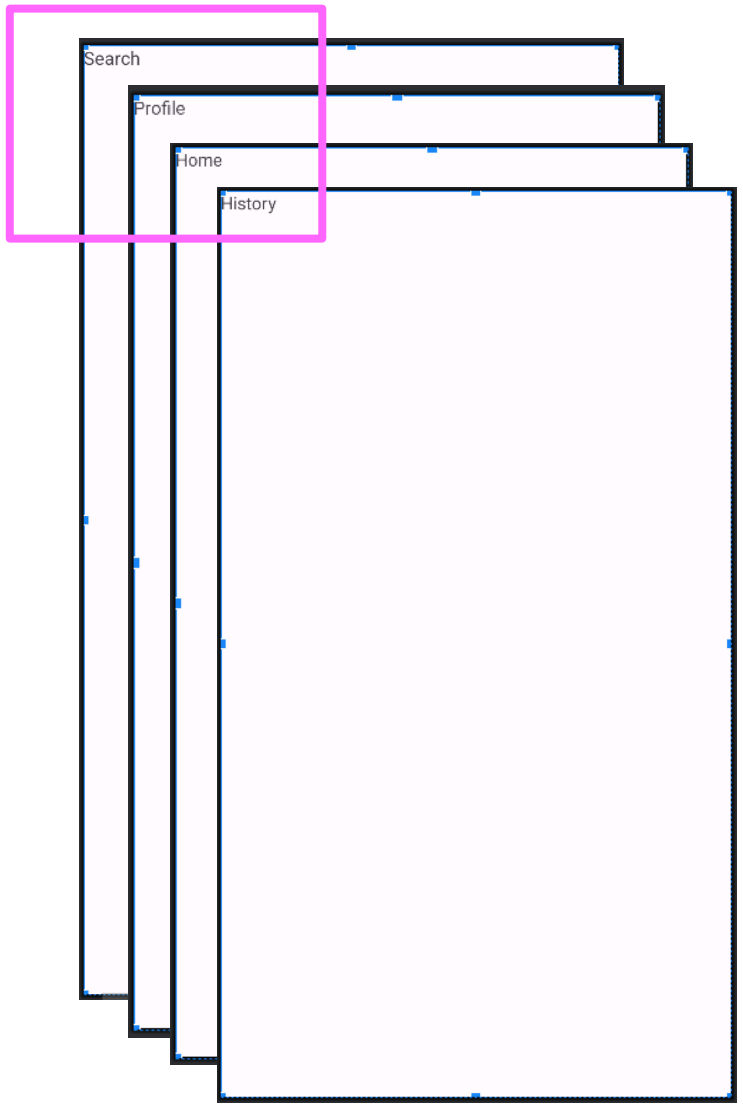
- Assign id same as fragment ids

```
<?xml version="1.0" encoding="utf-8"?>
<menu xmlns:android="http://schemas.android.com/apk/res/android">

    <item android:title="Home" android:icon="@drawable/home" android:id="@+id/homeFragment" />
    <item android:title="Cart"  android:icon="@drawable/cart" android:id="@+id/cartFragment"  />
    <item android:title="Search" android:icon="@drawable/search" android:id="@+id/searchFragment" />
    <item android:title="History" android:icon="@drawable/history"  android:id="@+id/historyFragment" />
    <item android:title="Profile" android:icon="@drawable/user" android:id="@+id/profileFragment" />
</menu>
```

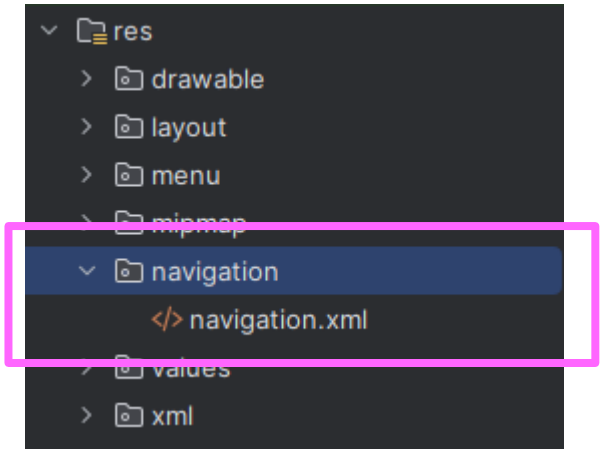
Fragment View titles

- Go to each fragment file
  res/layout/fragment***.xml
- Change text to appropriate fragment title
- Cart, home, history etc.
- So that when fragment is switched, we can see the difference.



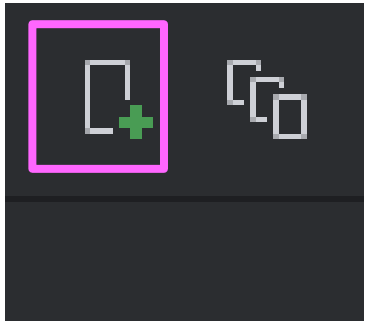
Navigation

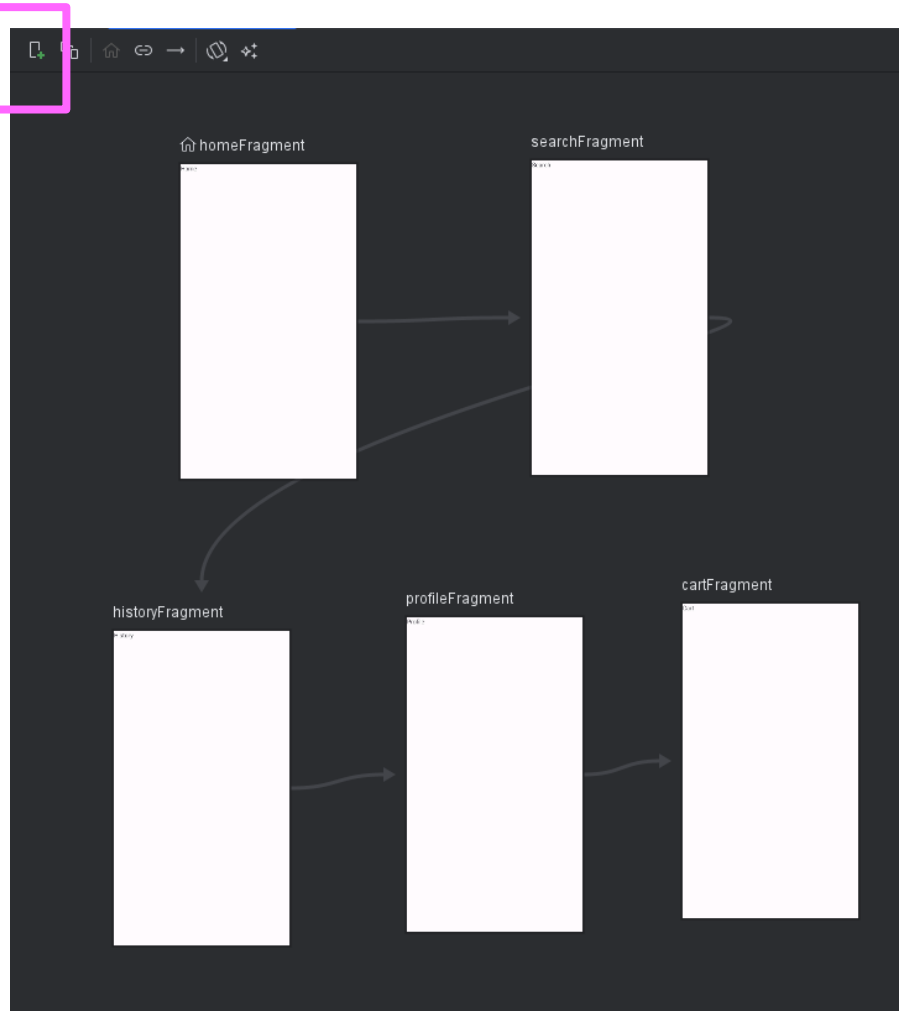
- Res: right click add Android Resource Directory
  - Name : navigation
  - Type: navigation
  - OK
- Navigation: right click add resource file
  - Name: navigation
  - OK



Add Fragment to Navigation

- Open res/navigation/navigation.xml
- Add fragments one by one
- Home, Cart, History, Search and Profile Fragments

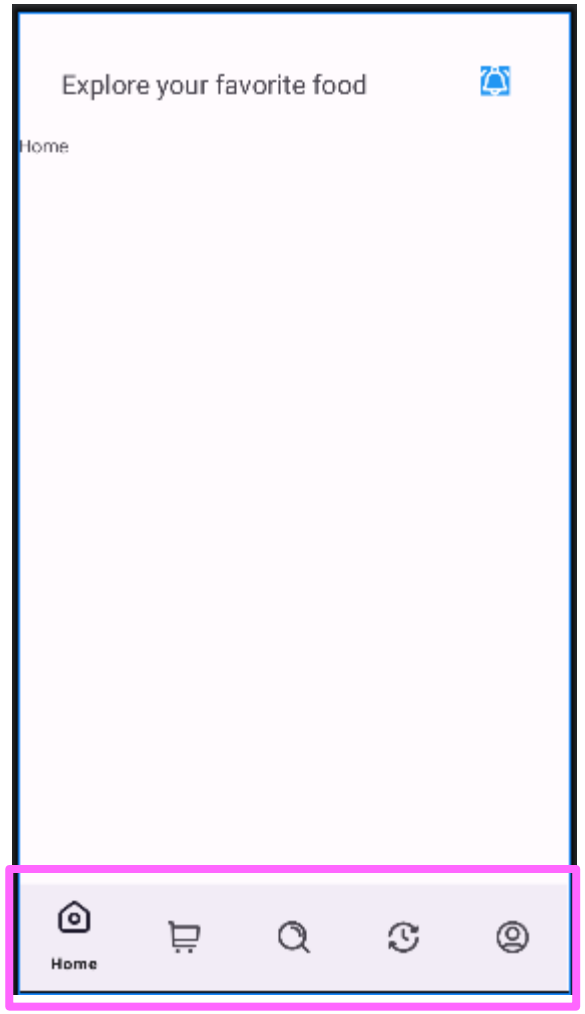




activity_main.xml : Add Navigation

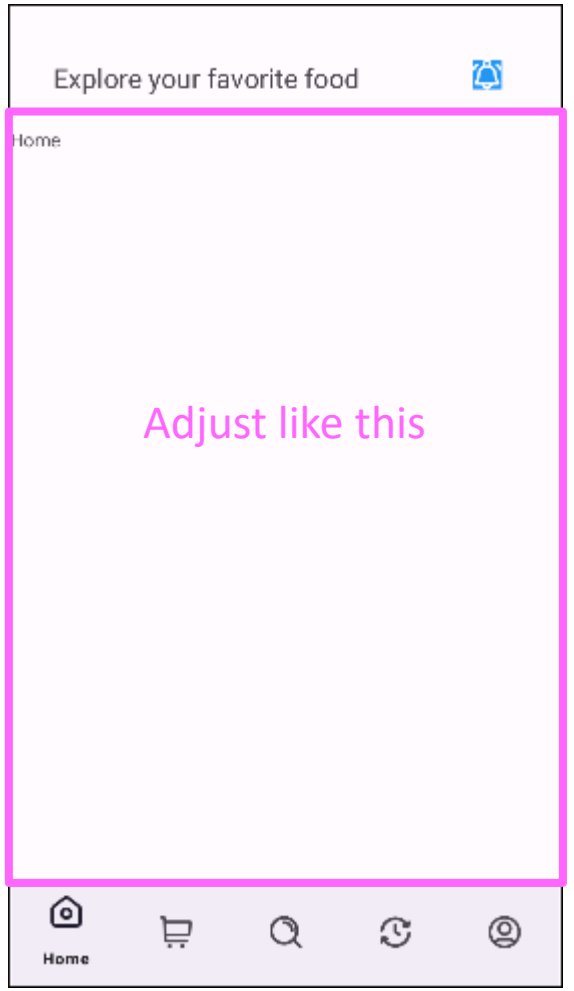
- Activity Main:
  - Design view
    - Add BottomNavigationView to the view
    - Select @menu/menu
    - Ok
  -

```
<com.google.android.material.bottomnavigation.BottomNavigationView
    android:id="@+id/bottomNavigationView"
    android:layout_width="409dp"
    android:layout_height="wrap_content"
    app:layout_constraintBottom_toBottomOf="parent"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintStart_toStartOf="parent"
    app:menu="@menu/menu" />
```



activity_main.xml : NavHostFragment

- From the Control Pallet
  - Add **NavHostFragment**
  - Select @navigation/navigation
  - Adjust layout to fit inside area

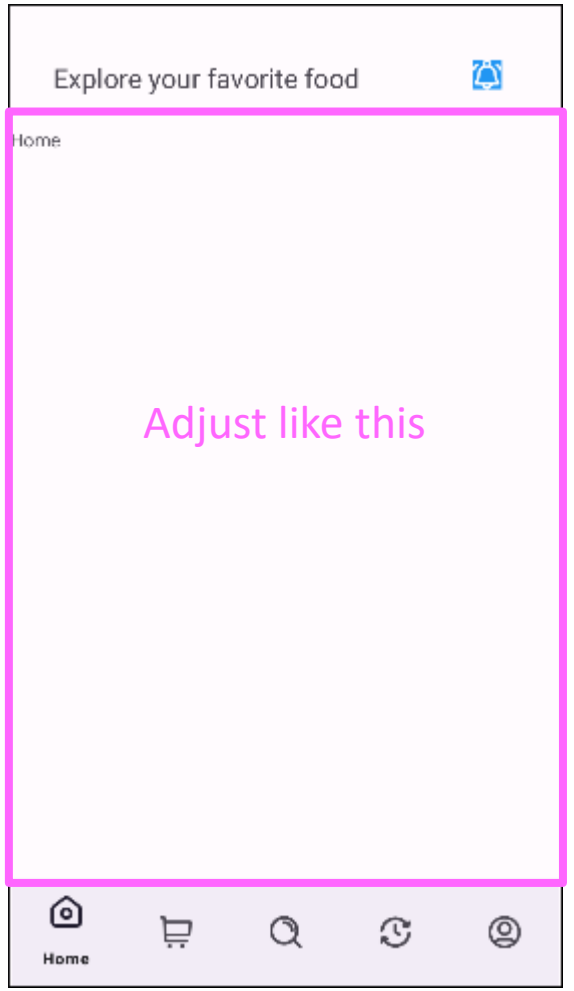


activity_main.xml : NavHostFragment

- In the xml code
- Change tag name from <…fragmentContainerView to <fragment

```
<androidx.fragment.app.FragmentContainerView
    android:id="@+id/fragmentContainerView"
    android:name="androidx.navigation.fragment.NavHostFragment"
    android:layout_width="0dp"
    android:layout_height="0dp"
    android:layout_marginTop="50dp"
    app:defaultNavHost="true"
    app:layout_constraintBottom_toBottomOf="parent"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toTopOf="@+id/textView"
    app:navGraph="@navigation/navigation" />
```

```
<fragment
    android:id="@+id/fragmentContainerView"
    android:name="androidx.navigation.fragment.NavHostFragment"
    android:layout_width="0dp"
    android:layout_height="0dp"
    android:layout_marginTop="50dp"
    app:defaultNavHost="true"
    app:layout_constraintBottom_toBottomOf="parent"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toTopOf="@+id/textView"
    app:navGraph="@navigation/navigation" />
```

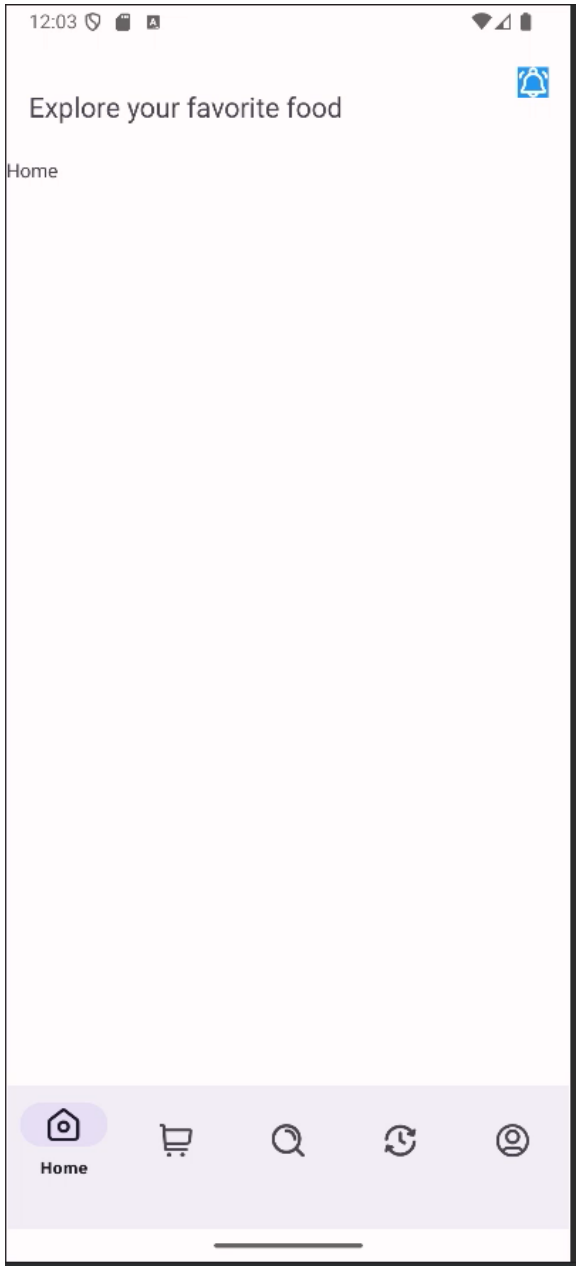


Code: MainActivity.kts Kotlin file

```
var navController=findNavController(R.id.fragmentContainerView)
var bottomNav=findViewById<BottomNavigationView>(R.id.bottomNavigationView)
bottomNav.setupWithNavController(navController)
```

Run and Test the work.

- Click the menu icons on the bottom navigation
- They should change the fragments inner area



End